

Module 1: Introduction to Geometric Learning

Section 1.3 — Manifold Learning

Data Science Master's Programme

Contents

1	The Manifold Hypothesis	2
2	Estimating the Intrinsic Dimension	2
2.1	Global linear estimate: PCA spectrum	3
2.2	Correlation dimension (Grassberger–Procaccia)	3
2.3	Maximum likelihood estimation (Levina–Bickel)	4
2.4	TwoNN: dimension from two nearest neighbours	4
2.5	The role of scale	5
2.6	Why intrinsic dimension matters for ML	5
2.7	Practical workflow	6
3	Isomap	6
3.1	Algorithm	6
3.2	The MDS Step in Detail	7
3.3	Limitations	7
4	Locally Linear Embedding (LLE)	7
4.1	Key Idea	7
4.2	Algorithm	7
4.3	Comparison with Isomap	8
5	Diffusion Maps	8
5.1	Construction	8
5.2	The role of t	9
6	Laplacian Eigenmaps	9
6.1	The graph Laplacian	10
6.2	Key properties of L	10
6.3	The Laplacian Eigenmaps algorithm	11
6.4	Variational interpretation	11
6.5	The continuum limit: the Laplace–Beltrami operator	11
6.6	Relationships with other methods	12
6.7	Practical notes	13
7	Modern Manifold Learning: t-SNE and UMAP	13
7.1	t -SNE: probabilistic neighbourhood preservation	13
7.2	The mass-of-clusters illusion	16
7.3	Pitfalls and goodness-of-trust	16
7.4	UMAP: a Riemannian-geometry-flavoured alternative	17
7.5	Variants and extensions of UMAP	20

7.6	Critical perspectives	20
7.7	The unifying perspective	21
8	Summary and Comparison	22

Motivation

Real-world datasets are nominally high-dimensional: a 256×256 colour image lives in $\mathbb{R}^{196\,608}$. Yet images of natural scenes are not randomly scattered throughout this space — they cluster around some much lower-dimensional structure. *Manifold learning* is the study of methods that discover and exploit this structure.

1 The Manifold Hypothesis

Definition 1.1: Manifold (intuitive)

A d -dimensional **manifold** $\mathcal{M}$ is a topological space that locally looks like $\mathbb{R}^d$. More precisely, every point $p \in \mathcal{M}$ has a neighbourhood that can be continuously and invertibly mapped (a *homeomorphism*) to an open subset of $\mathbb{R}^d$.

Classic examples:

- The circle $S^1 \subset \mathbb{R}^2$: a 1D manifold.
- The sphere $S^2 \subset \mathbb{R}^3$: a 2D manifold.
- The surface of a torus: a 2D manifold.

Definition 1.2: The Manifold Hypothesis

High-dimensional data (images, text embeddings, audio, etc.) approximately lie on a low-dimensional manifold $\mathcal{M} \subset \mathbb{R}^D$, where the *intrinsic dimension* $d \ll D$.

Example 1.1: Image manifolds

Consider the set of all 28×28 images of handwritten digit “3.” This set lives in $\mathbb{R}^{784}$, but its intrinsic degrees of freedom are few: thickness of stroke, slant, size, position. Empirically, such datasets have intrinsic dimension $d \approx 10$ –20.

Consequence for distance. If data lie on a curved manifold, the straight-line (Euclidean) distance $\|x - y\|$ between two points can be misleading. What matters geometrically is the *geodesic distance*: the length of the shortest path *along the manifold* connecting x and y .

Example 1.2: Geodesic vs Euclidean distance

Imagine data lying on a Swiss roll: a 2D sheet rolled up in 3D. Two points near each other in $\mathbb{R}^3$ (short Euclidean distance) may be far apart along the roll (large geodesic distance), and vice versa. A clustering or dimensionality-reduction method based on Euclidean distances will therefore give poor results.

2 Estimating the Intrinsic Dimension

Before applying manifold learning, a fundamental question must be answered: *what is the intrinsic dimension d of the data?* The manifold hypothesis only tells us that $d \ll D$ — it does not say what d is. This number governs every downstream decision: the target embedding dimension, the choice of k in nearest-neighbour graphs, the bandwidth of kernels, and ultimately how much distortion to expect in any low-dimensional visualisation.

Definition 2.1: Intrinsic dimension

Let $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^D$ be a finite dataset. Its **intrinsic dimension** d is the dimension of the smallest (locally Euclidean) manifold $\mathcal{M}$ that approximately contains X . Equivalently, d is the smallest number of *independent degrees of freedom* needed to parametrise the data.

This definition is intentionally fuzzy: real data only *approximately* lies on a manifold, the local dimension may vary across $\mathcal{M}$, and the answer often depends on the scale at which we look. The estimators below give increasingly refined answers.

2.1 Global linear estimate: PCA spectrum

The simplest estimator: run PCA and count how many principal components carry significant variance.

- Compute the covariance $\Sigma = \frac{1}{n} \sum_i (x_i - \bar{x})(x_i - \bar{x})^\top$.
- Diagonalise: $\Sigma = V\Lambda V^\top$ with $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_D$.
- Choose d such that $\sum_{i=1}^d \lambda_i / \sum_{i=1}^D \lambda_i \geq 0.95$ (the 95% variance rule), or look for an “elbow” in the spectrum.

Example 2.1: PCA spectrum of a Swiss roll

A Swiss roll embedded in $\mathbb{R}^3$ has *intrinsic* dimension 2 (it is a folded 2-D sheet). PCA, however, sees it spread across all three ambient dimensions and returns $d \approx 3$. This is the classic failure mode of PCA on curved data: *linear estimators inflate intrinsic dimension whenever the manifold is curved*, because curvature requires extra ambient directions to represent.

PCA is fast and stable, but gives an *upper bound* on d that is often loose. For non-linear manifolds we need non-linear estimators.

2.2 Correlation dimension (Grassberger–Procaccia)

A staple of dynamical-systems analysis (Grassberger and Procaccia, 1983). The idea: count the number of point pairs closer than a distance r and see how this count scales with r .

Definition 2.2: Correlation integral

$$C(r) = \frac{2}{n(n-1)} \sum_{i < j} \mathbf{1}\{\|x_i - x_j\| < r\}.$$

For points uniformly sampled from a d -dimensional manifold, $C(r) \sim r^d$ as $r \rightarrow 0$ (small enough that the manifold looks locally Euclidean, but large enough to escape sampling noise).

The **correlation dimension** is then

$$d_{\text{corr}} = \lim_{r \rightarrow 0} \frac{\log C(r)}{\log r}.$$

Practical estimation. Plot $\log C(r)$ versus $\log r$ on a log–log axis. There is typically a roughly linear “scaling regime” for intermediate r ; the slope of the regression line in this regime estimates d . At very small r noise dominates (slope $\rightarrow 0$); at very large r the finite size of the dataset saturates the count (slope $\rightarrow 0$ again).

Limitations. The choice of scaling regime is subjective; the estimator has high variance for small datasets; it assumes a uniform sampling density. For low-dimensional manifolds it remains a useful benchmark.

2.3 Maximum likelihood estimation (Levina–Bickel)

Levina and Bickel (2004) derive an estimator from a Poisson-process model of the nearest-neighbour structure. For each point x_i , let $T_k(x_i)$ be the distance from x_i to its k -th nearest neighbour. Under the assumption that points are sampled i.i.d. from a d -dimensional manifold with smooth density, the maximum likelihood estimate of the local dimension at x_i , using its k nearest neighbours, is

$$\hat{d}_k(x_i) = \left[\frac{1}{k-1} \sum_{j=1}^{k-1} \log \frac{T_k(x_i)}{T_j(x_i)} \right]^{-1}.$$

Derivation sketch. Inside a small ball of radius r around x_i , the number of sample points scales as $N(r) \sim r^d$. The likelihood of observing the specific sequence of nearest-neighbour distances $T_1, T_2, \dots, T_{k-1}$ given the model gives the formula above after maximising over d (with $\log T_k/T_j$ playing the role of the sufficient statistic).

Combining local estimates. Levina and Bickel originally averaged $\hat{d}_k(x_i)$ over i , which turns out to be biased. MacKay and Ghahramani (2005) showed that the *harmonic* (inverse) average is preferable:

$$\hat{d}_k^{\text{MG}} = \left[\frac{1}{n} \sum_{i=1}^n \hat{d}_k(x_i)^{-1} \right]^{-1}.$$

Choosing k . Small k (5–10) probes truly local geometry but is noise-sensitive; large k (50–100) averages out noise but may cross the curvature scale of the manifold and underestimate d . A common practice is to plot $\hat{d}_k$ as a function of k and look for a plateau.

2.4 TwoNN: dimension from two nearest neighbours

Facco, d’Errico, Rodriguez, and Laio (2017) introduced an even simpler estimator: **TwoNN**, which uses only the two nearest neighbours of each point.

Definition 2.3: TwoNN ratio

For each x_i , let $r_1(x_i)$ and $r_2(x_i)$ be the distances to its first and second nearest neighbours. Define the ratio

$$\mu_i = r_2(x_i)/r_1(x_i) \geq 1.$$

Theorem 2.1: Distribution of μ for uniform sampling

Under uniform sampling from a d -dimensional manifold with smooth density, μ_i follows a Pareto distribution with shape d :

$$\Pr(\mu \geq t) = t^{-d}, \quad t \geq 1.$$

Equivalently, $\log \mu_i \sim \text{Exp}(d)$, so

$$\hat{d}_{\text{TwoNN}} = \frac{n}{\sum_{i=1}^n \log \mu_i}.$$

Why this works. The probability that the second nearest neighbour is at distance $\geq t \cdot r_1$ is the probability that the spherical shell between r_1 and tr_1 contains no sample point. For a d -dimensional manifold this shell has volume proportional to $(t^d - 1)r_1^d$ near x_i , leading to the Pareto law after marginalising over r_1 .

Robustness. TwoNN has three appealing properties:

- **Locality.** Using only r_1 and r_2 keeps the estimate at the smallest accessible scale; curvature has minimal effect.
- **Density-independence.** The ratio $\mu = r_2/r_1$ cancels the local density, so the estimator works on non-uniformly sampled data.
- **Goodness-of-fit check.** Plotting the empirical CDF of $\log \mu_i$ on a log scale should give a straight line of slope $-d$; deviations from linearity flag violations of the underlying assumptions (multi-scale manifolds, noisy data).

Limitations. TwoNN assumes that the local dimension is constant across the dataset. Real data — e.g. unions of manifolds of different dimensions, or hierarchical data — violates this. Variants such as *GRIDE* (Denti et al., 2022) extend the estimator to use further neighbours adaptively.

2.5 The role of scale

Real datasets often have a *scale-dependent* intrinsic dimension:

- **Sub-noise scale** (r very small): noise is full-dimensional, so $\hat{d}(r) \rightarrow D$ as $r \rightarrow 0$.
- **Manifold scale** (r at the typical inter-point distance): $\hat{d}(r) \approx d$, the true intrinsic dimension.
- **Global scale** (r comparable to dataset size): curvature dominates, and $\hat{d}(r)$ inflates above the true d as the algorithm sees the manifold’s extrinsic embedding.

Plotting $\hat{d}$ as a function of scale (the *dimension trajectory*) is therefore informative: a plateau indicates a clean manifold scale; multiple plateaus indicate a hierarchical structure where small-scale features (noise, fine-grained features) and large-scale features (coarse classes) have different effective dimensions.

2.6 Why intrinsic dimension matters for ML

Remark 2.1: Curse of dimensionality and the rescue

Many learning-theoretic bounds (sample complexity, convergence rates for non-parametric methods) scale as $n^{-c/d}$ or worse. If the ambient dimension $D = 10^4$ but the intrinsic dimension is $d = 10$, learning is fundamentally tractable. If the intrinsic dimension is also $\sim D$, generalisation requires either explicit regularisation or extra structural priors (such as the symmetries of Section 1.2).

Example 2.2: Intrinsic dimension of natural images and language

Pope, Zhu, Abdelkader, Goldblum, and Goldstein (2021) used TwoNN-type estimators to measure the intrinsic dimension of standard ML datasets:

Dataset	Ambient D	Intrinsic d (estimated)
MNIST	784	≈ 12 –14
CIFAR-10	3072	≈ 25 –35
ImageNet	$\sim 150,000$	≈ 40 –45
Natural language (BERT embeddings)	768	≈ 10 –20

The intrinsic dimension explains why deep neural networks generalise despite their nominally enormous capacity: the data they must learn to handle is concentrated near a much lower-dimensional manifold than the ambient pixel space suggests.

2.7 Practical workflow

A reasonable workflow before doing manifold learning or training a non-parametric model:

1. Run PCA and inspect the spectrum — get a rough *upper bound* on d .
2. Run TwoNN — get a robust local estimate of d at the smallest scale.
3. Run Levina–Bickel for several values of k — verify the plateau.
4. If the three estimates agree, you have a reliable d . If they disagree, the data has scale-dependent structure that warrants further analysis before reducing dimension.

Remark 2.2: From intrinsic dimension to embedding dimension

The intrinsic dimension d is a *lower bound* on the embedding dimension you should choose for downstream tasks: you cannot embed in fewer than d dimensions without throwing away geometric information. For *visualisation* we accept the distortion of embedding in $\mathbb{R}^2$ even when d is much larger — this is the cost of human-readable plots. For *representation learning* pipelines (e.g. as input features to a classifier), aim for embedding dimensions of at least $2d$ – $3d$ to leave room for the manifold’s curvature.

3 Isomap

Isomap (Tenenbaum, de Silva, Langford, 2000) was one of the first algorithms to replace Euclidean distances with approximated geodesic distances before performing dimensionality reduction.

3.1 Algorithm

1. **Build a neighbourhood graph.** For each point x_i , connect it to its k nearest neighbours (or all points within radius ε). Edge weights are Euclidean distances $\|x_i - x_j\|$.
2. **Compute geodesic distances.** Use Dijkstra's or Floyd–Warshall's algorithm to find the shortest-path distance $d_G(x_i, x_j)$ between all pairs of points in the graph. This approximates the true geodesic distance on the manifold.
3. **Classical MDS.** Apply *multidimensional scaling* (MDS) to the geodesic distance matrix to find a d -dimensional embedding that preserves these distances as well as possible. MDS centres and double-centres the squared distance matrix, then takes the top- d eigenvectors.

3.2 The MDS Step in Detail

Given a distance matrix D with entries $D_{ij} = d_G(x_i, x_j)$, form the matrix B by *double centering*:

$$B_{ij} = -\frac{1}{2} \left(D_{ij}^2 - \frac{1}{n} \sum_k D_{ik}^2 - \frac{1}{n} \sum_k D_{kj}^2 + \frac{1}{n^2} \sum_{k,l} D_{kl}^2 \right).$$

If the distances are Euclidean, B is positive semi-definite and $B = ZZ^\top$ where $Z \in \mathbb{R}^{n \times d}$ is the desired embedding. In general we take the eigendecomposition $B \approx V\Lambda V^\top$ (keeping only positive eigenvalues) and set $Z = V\Lambda^{1/2}$.

Theorem 3.1: Correctness of Isomap on flat manifolds

If the data lie exactly on a flat d -dimensional subspace of $\mathbb{R}^D$ and the neighbourhood graph is connected, Isomap recovers the original d -dimensional coordinates exactly (up to rotation and reflection).

3.3 Limitations

- **Topology:** Isomap assumes the manifold is *simply connected* (no holes). It can fail on data shaped like a ring or torus.
- **Noise:** Short-circuit edges (connecting distant regions via noisy shortcuts) distort geodesic estimates severely.
- **Scalability:** All-pairs shortest paths costs $O(n^2 \log n)$ time and $O(n^2)$ space.
- **Out-of-sample:** Embedding a new point requires solving an additional optimisation problem (or using the Nyström approximation).

4 Locally Linear Embedding (LLE)

LLE (Roweis and Saul, 2000) takes a different approach: instead of computing global geodesic distances, it exploits *local linear structure*.

4.1 Key Idea

On a smooth manifold, each point can be approximately expressed as a *linear combination of its neighbours*:

$$x_i \approx \sum_{j \in \mathcal{N}(i)} w_{ij} x_j,$$

where $\mathcal{N}(i)$ is the set of k nearest neighbours of x_i . The weights w_{ij} capture the local geometry. LLE finds an embedding that *preserves these weights* in the low-dimensional space.

4.2 Algorithm

1. **Find neighbours.** For each x_i , identify its k nearest neighbours $\{x_j\}_{j \in \mathcal{N}(i)}$.
2. **Compute reconstruction weights.** Solve the constrained least-squares problem

$$\min_{w_i} \left\| x_i - \sum_{j \in \mathcal{N}(i)} w_{ij} x_j \right\|^2 \quad \text{subject to} \quad \sum_j w_{ij} = 1.$$

The constraint makes the weights invariant to rotation, scaling, and translation of the neighbourhood.

3. **Compute low-dimensional embedding.** Find coordinates $y_i \in \mathbb{R}^d$ that minimise the same reconstruction error with the *same weights*:

$$\min_{\{y_i\}} \sum_i \left\| y_i - \sum_{j \in \mathcal{N}(i)} w_{ij} y_j \right\|^2.$$

This reduces to finding the eigenvectors of a sparse $n \times n$ matrix (the bottom $d + 1$ eigenvectors, discarding the trivial one).

4.3 Comparison with Isomap

Aspect	Isomap	LLE
Distance type	Global geodesic	Local linear
Handles holes	No	Yes (locally)
Noise robustness	Moderate	Moderate
Matrix to factorise	Dense $n \times n$	Sparse $n \times n$
Assumes	Convex manifold	Locally linear

5 Diffusion Maps

Diffusion Maps (Coifman and Lafon, 2006) take a probabilistic viewpoint: they model the data as a *Markov chain on a graph* and use the geometry of random walk diffusion to define distances.

5.1 Construction

1. **Build a kernel matrix.** Define a pairwise affinity (similarity) using a Gaussian kernel:

$$k(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right).$$

2. **Normalise to a Markov matrix.** Let $d_i = \sum_j k(x_i, x_j)$. Form the row-stochastic matrix (transition probabilities):

$$P_{ij} = \frac{k(x_i, x_j)}{d_i}.$$

P describes a random walk: from x_i , jump to x_j with probability P_{ij} .

3. **Define diffusion distance.** After t steps of the random walk, the matrix P^t describes the distribution of the walker. The **diffusion distance** at time t is:

$$D_t(x_i, x_j)^2 = \sum_k \frac{(P_{ik}^t - P_{jk}^t)^2}{\phi_0(k)},$$

where ϕ_0 is the stationary distribution. Two points are close if a random walker started from one is likely to be found near the other after t steps.

4. **Spectral embedding.** Let $\lambda_1 \geq \lambda_2 \geq \dots$ be the eigenvalues of P (real numbers in $[-1, 1]$ since P is row-stochastic) and $\psi_1, \psi_2, \dots$ the corresponding right eigenvectors, with $\psi_k(i)$ denoting the i -th component of ψ_k . Fix a diffusion time $t > 0$ (a positive real number, typically a small integer). The **diffusion map** at time t embeds x_i as

$$\Psi_t(x_i) = (\lambda_1^t \psi_1(i), \lambda_2^t \psi_2(i), \dots, \lambda_d^t \psi_d(i)) \in \mathbb{R}^d.$$

Notation: λ_k^t denotes the scalar eigenvalue λ_k raised to the (real) power t — *not* a transpose. Each component of $\Psi_t(x_i)$ is the product of a scalar λ_k^t and a scalar $\psi_k(i)$.

Theorem 5.1: Diffusion distance and embedding

The Euclidean distance in the diffusion map embedding equals the diffusion distance:

$$\|\Psi_t(x_i) - \Psi_t(x_j)\|^2 = D_t(x_i, x_j)^2.$$

5.2 The role of t

The parameter t controls the *scale* of the geometry explored:

- Small t : local geometry (noise-sensitive).
- Large t : global, coarse-grained structure (small eigenvalues $\lambda_k^t \approx 0$ are automatically suppressed, acting as a denoiser).

Example 5.1: Diffusion Maps for single-cell RNA sequencing

In single-cell genomics, each cell is a point in $\mathbb{R}^D$ (gene expression space, $D \approx 20\,000$). Diffusion Maps are routinely used to visualise differentiation trajectories: cells at similar stages of development land close together in the diffusion embedding, revealing the branching structure of cellular lineages. The parameter t is chosen so that transient (noisy) connections are suppressed while developmental paths remain connected.

Remark 5.1: Diffusion Maps inherit the manifold's geometry

As $n \rightarrow \infty$ and the kernel bandwidth $\sigma \rightarrow 0$ in a coordinated way, the random-walk operator P converges to the semigroup $e^{-t\Delta_{\mathcal{M}}}$ generated by the *Laplace–Beltrami operator* of the underlying manifold (Coifman & Lafon, 2006). Diffusion Maps are therefore not just a graph algorithm: they are a finite-sample approximation of a fundamental object in differential geometry. This sets up the next section, where the same operator is approached directly.

6 Laplacian Eigenmaps

Laplacian Eigenmaps (Belkin and Niyogi, 2003) is the *purest* spectral manifold-learning method: it computes the eigenvectors of the graph Laplacian directly, without first building a Markov chain or computing geodesic distances. The method has deep connections to spectral graph theory, to the heat equation on manifolds, and — through these — to graph neural networks.

6.1 The graph Laplacian

Given the data, build a weighted neighbourhood graph (the same step as Isomap or Diffusion Maps). Edge weights W_{ij} can be binary ($W_{ij} = 1$ if j is among x_i 's k nearest neighbours, else 0) or use a heat kernel:

$$W_{ij} = \exp\left(-\|x_i - x_j\|^2/2\sigma^2\right).$$

Definition 6.1: Graph Laplacian

Let $W \in \mathbb{R}^{n \times n}$ be a symmetric non-negative weight matrix on n points and let $D = \text{diag}(d_1, \dots, d_n)$ be the diagonal degree matrix with $d_i = \sum_j W_{ij}$. The **unnormalised graph Laplacian** is

$$L = D - W.$$

Common normalised variants are

$$L_{\text{sym}} = D^{-1/2} L D^{-1/2} = I - D^{-1/2} W D^{-1/2} \quad (\text{symmetric normalised}),$$

$$L_{\text{rw}} = D^{-1} L = I - D^{-1} W \quad (\text{random-walk normalised}).$$

Remark 6.1: Why these names?

The matrix $D^{-1}W$ is the Markov matrix P from the diffusion-maps section: it is the row-stochastic transition matrix of a random walk on the graph. So $L_{\text{rw}} = I - P$ — the random-walk Laplacian is just one minus the diffusion operator. This is the formal link between Laplacian Eigenmaps and Diffusion Maps: the same matrix, viewed through different spectral lenses.

6.2 Key properties of L

The graph Laplacian has a remarkable amount of structure encoded in its spectrum:

Theorem 6.1: Properties of the unnormalised Laplacian

Let $L = D - W$ be the unnormalised Laplacian of a graph with symmetric non-negative weights. Then:

1. L is symmetric and positive semi-definite. All eigenvalues are real and non-negative.
2. The smallest eigenvalue is 0, with eigenvector the all-ones vector $\mathbf{1}$ (one 0-eigenvalue per connected component of the graph).
3. For any $f \in \mathbb{R}^n$,

$$f^\top L f = \frac{1}{2} \sum_{i,j} W_{ij} (f_i - f_j)^2.$$

This is the **Dirichlet energy** of f on the graph: it is small when f varies slowly along high-weight edges.

Reading the theorem. Property 3 is the heart of Laplacian Eigenmaps. The eigenvectors of L associated with small eigenvalues are precisely the functions on the graph that vary as little as possible along the graph edges — they are the “smoothest” coordinates on the data. Using these as embedding coordinates preserves the local neighbourhood structure: if W_{ij} is large, the embedded coordinates y_i and y_j are encouraged to be close.

6.3 The Laplacian Eigenmaps algorithm

1. **Build the neighbourhood graph.** For each x_i , find its k nearest neighbours; set W_{ij} via heat kernel or binary indicator.
2. **Form the Laplacian.** Compute D and choose L , L_{sym} , or L_{rw} (the choice affects the geometry the embedding inherits — see below).
3. **Solve the generalised eigenproblem.** Find the smallest $d + 1$ eigenvalue/eigenvector pairs (λ_i, ψ_i) of

$$L\psi = \lambda D\psi.$$

Equivalently, find the smallest eigenvectors of L_{rw} or L_{sym} .

4. **Discard the trivial eigenvector.** The smallest eigenvalue is $\lambda_0 = 0$, with constant eigenvector $\psi_0 \propto \mathbf{1}$. This carries no embedding information.
5. **Embed.** Use the next d eigenvectors as embedding coordinates:

$$y_i = (\psi_1(i), \psi_2(i), \dots, \psi_d(i)) \in \mathbb{R}^d.$$

6.4 Variational interpretation

Theorem 6.2: Laplacian Eigenmaps optimality

The Laplacian Eigenmaps embedding $Y = [y_1, \dots, y_n]^\top$ minimises the *embedding distortion*

$$\mathcal{E}(Y) = \sum_{i,j} W_{ij} \|y_i - y_j\|^2 = \text{tr}(Y^\top LY)$$

subject to the orthogonality constraint $Y^\top DY = I$ (which removes the trivial constant solution and prevents collapse).

In words: *find a low-dimensional embedding in which neighbouring points stay close* (weighted by their graph affinity), normalised so that the embedding spreads its mass across the dataset rather than collapsing to a point.

Why the constraint? Without normalisation, the minimisation trivially returns $Y = 0$ or $Y = \mathbf{1}$. The constraint $Y^\top DY = I$ ensures that the embedding is non-trivial *and* that points of high degree (well-connected vertices) get correspondingly larger “vote” in setting the global scale.

Solution via the generalised eigenproblem. By Lagrangian arguments, the constrained minimum is achieved by the bottom non-trivial eigenvectors of the generalised eigenproblem $L\psi = \lambda D\psi$. This is why Laplacian Eigenmaps reduces to a spectral computation.

6.5 The continuum limit: the Laplace–Beltrami operator

The graph Laplacian is a discretisation of a fundamental object in differential geometry.

Definition 6.2: Laplace–Beltrami operator

For a smooth function $f : \mathcal{M} \rightarrow \mathbb{R}$ on a Riemannian manifold $(\mathcal{M}, g)$, the **Laplace–Beltrami operator** is

$$\Delta_{\mathcal{M}} f = -\frac{1}{\sqrt{\det g}} \sum_{i,j} \partial_i \left(\sqrt{\det g} g^{ij} \partial_j f \right),$$

where g_{ij} is the metric tensor and g^{ij} its inverse. In flat $\mathbb{R}^d$ with the Euclidean metric, $\Delta = -\sum_i \partial_i^2$ — the familiar Laplacian.

Theorem 6.3: Convergence to the Laplace–Beltrami operator

Suppose n points are sampled i.i.d. from a uniform distribution on a smooth d -dimensional manifold $\mathcal{M} \subset \mathbb{R}^D$, with heat-kernel weights $W_{ij} = e^{-\|x_i - x_j\|^2 / 2\sigma^2}$. Then, as $n \rightarrow \infty$ and $\sigma \rightarrow 0$ in a coordinated way ($n\sigma^{d+2} \rightarrow \infty$), the graph Laplacian L_{rw} , suitably rescaled, converges (pointwise on smooth test functions) to the Laplace–Beltrami operator $\Delta_{\mathcal{M}}$ (Belkin & Niyogi, 2008; Singer, 2006).

Remark 6.2: Interpretation

The eigenvectors of L on a sufficiently dense sample of $\mathcal{M}$ approximate the *eigenfunctions* of $\Delta_{\mathcal{M}}$ on the underlying manifold. These eigenfunctions are the manifold’s intrinsic “frequencies” — the analogue of Fourier basis functions for the (curved) manifold. Using them as embedding coordinates is the natural manifold-generalisation of mapping data into a Fourier basis.

6.6 Relationships with other methods

Laplacian Eigenmaps occupies a central spot in the family of spectral manifold-learning methods:

- **Diffusion Maps** compute eigenvectors of the same matrix $P = D^{-1}W$, but use *powered* eigenvalues λ_k^t for time-controlled multi-scale embedding. Laplacian Eigenmaps is the $t = 1$ case modulo a trivial eigenvalue rescaling. Diffusion Maps adds the diffusion-time knob t ; Laplacian Eigenmaps fixes it implicitly.
- **LLE** produces embeddings that, in the appropriate limit, also approximate the eigenfunctions of a manifold-dependent operator (the *Hessian* on the manifold, Donoho & Grimes 2003). LLE and Laplacian Eigenmaps are therefore close cousins; in practice their outputs are often qualitatively similar.
- **Spectral clustering** (Shi & Malik 2000; Ng, Jordan, Weiss 2001) uses *exactly* the same eigenvectors of L , but interprets them differently: instead of as embedding coordinates, they are clustered (e.g. with k -means) to partition the data into clusters. Laplacian Eigenmaps and spectral clustering are two sides of the same spectral coin.
- **Graph Neural Networks.** The convolutional layer of a *Graph Convolutional Network* (Kipf & Welling 2017) is a polynomial in the normalised Laplacian L_{sym} . Laplacian Eigenmaps is therefore the unsupervised feature construction that GCNs use *implicitly* via their spectral architecture. We will pursue this connection in Module 2.

- **Heat-kernel signatures** in computer graphics use the same eigendecomposition to produce shape descriptors on meshes. Sun, Ovsjanikov & Guibas (2009) define $\text{HKS}(x, t) = \sum_k e^{-\lambda_k t} \psi_k(x)^2$ — directly the diagonal of the heat kernel built from the Laplacian eigendecomposition.

6.7 Practical notes

- **Computation.** The bottom d eigenvectors of a sparse $n \times n$ matrix can be found in $O(n \cdot d \cdot k)$ time using Lanczos iteration. Scales to $n \sim 10^5$ comfortably on a workstation; for larger problems use Nyström subsampling.
- **Choice of Laplacian.** L_{sym} is symmetric (numerically more stable than L_{rw}), but its eigenvectors are rescaled versions of L_{rw} 's. L_{rw} is preferred when the underlying manifold is non-uniformly sampled, because its eigenvectors converge to the genuine Laplace–Beltrami eigenfunctions independently of density.
- **Connectivity.** The number of zero eigenvalues equals the number of connected components of the graph. Choose k large enough that the k -NN graph is connected, or restrict the analysis to the largest component.
- **Bandwidth σ .** For heat-kernel weights, a common choice is $\sigma \approx$ the median pairwise distance, or an adaptive per-point bandwidth (analogous to t-SNE's perplexity).

Example 6.1: Eigenfunctions of the circle S^1

For n points uniformly sampled on S^1 , the eigenvectors of L approximate $\sin(k\theta)$ and $\cos(k\theta)$ for $k = 1, 2, 3, \dots$ — the *Fourier basis* on the circle. Embedding into $\mathbb{R}^2$ using ψ_1, ψ_2 recovers a circle topology faithfully: the embedding is a 2π -rotation of the original S^1 .

Remark 6.3: Laplacian Eigenmaps in single-cell genomics

Before UMAP became dominant, Laplacian Eigenmaps and Diffusion Maps were the standard tools for visualising single-cell RNA-seq data. They remain the methods of choice when global geometric correctness matters more than visual cluster separation, e.g. when inferring developmental *trajectories* (*Destiny*, Angerer et al., 2016; *Wishbone*, Setty et al., 2016).

7 Modern Manifold Learning: t -SNE and UMAP

The methods above (Isomap, LLE, Diffusion Maps) are *classical* manifold learning: they have closed-form spectral solutions and clean theoretical foundations. In practice, contemporary practitioners often reach instead for two non-spectral methods that prioritise visualisation quality and scalability over mathematical purity.

This section develops t -SNE and UMAP in enough depth to use them critically — including their hyperparameters, failure modes, and the crucial fact that *they are visualisation tools, not embeddings to build models on top of*.

7.1 t -SNE: probabilistic neighbourhood preservation

t -SNE (van der Maaten and Hinton, 2008) — *t-distributed Stochastic Neighbour Embedding* — is the descendant of SNE (Hinton and Roweis, 2002). Its key idea: model the *neighbourhood relationships* as probability distributions in both high- and low-dimensional space, then match them by minimising a KL divergence.

High-dimensional similarities. For each pair of points x_i, x_j , define a conditional probability that x_j is a neighbour of x_i :

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}.$$

The variance σ_i is set *per point* so that the entropy of the distribution $p_{\cdot|i}$ equals $\log_2(\text{perplexity})$. Symmetrise:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}.$$

Remark 7.1: Adaptive bandwidth and perplexity

Setting σ_i individually is what makes t -SNE adapt to varying local density: in dense regions σ_i shrinks, in sparse regions it grows. The hyperparameter *perplexity* (typical range 5–50) is a *soft* target for the effective number of neighbours of each point. Too small: t -SNE sees only nearest neighbours and produces many tiny disconnected blobs. Too large: distinct clusters merge.

Computing σ_i from perplexity. The relationship between bandwidth and perplexity is implicit: define the (Shannon) entropy of $p_{\cdot|i}$ as

$$H(p_{\cdot|i}) = - \sum_j p_{j|i} \log_2 p_{j|i},$$

and define $\text{perplexity}(p_{\cdot|i}) = 2^{H(p_{\cdot|i})}$. Given a user-chosen target perplexity, t -SNE performs a binary search over σ_i for each point until the achieved perplexity matches the target. Because H is monotone increasing in σ , the search converges within ~ 50 iterations per point.

Remark 7.2: Perplexity is roughly k in k -NN

A useful intuition: $\text{perplexity} \approx k$ where k is the effective number of nearest neighbours that the Gaussian kernel “sees” for each point. Setting perplexity to 30 means each point’s distribution $p_{\cdot|i}$ effectively concentrates its mass on ~ 30 neighbours. This is why the recommended default (30) makes sense for medium-sized datasets but should scale upward for very large n .

Low-dimensional similarities. In the embedding space (typically $\mathbb{R}^2$ for visualisation), similarities follow a Student- t distribution with one degree of freedom (i.e. a Cauchy):

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq \ell} (1 + \|y_k - y_\ell\|^2)^{-1}}.$$

Remark 7.3: Why a heavy tail?

The original SNE used Gaussians in both spaces, but this caused the “crowding problem”: because the volume of a ball at radius r grows as r^d , mapping a high- d manifold into 2D forces moderate distances to compress into a tiny annulus. The heavy tail of the Student- t (which has a single parameter and decays as $1/r^2$) lets moderate distances *spread out* in 2D, producing the well-separated clusters t -SNE is famous for.

The objective. Minimise the KL divergence

$$C(Y) = \text{KL}(P \| Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}},$$

by gradient descent on the embedding coordinates $Y = \{y_1, \dots, y_n\}$. The gradient has a clean form (van der Maaten and Hinton, 2008):

$$\frac{\partial C}{\partial y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij}) (y_i - y_j) (1 + \|y_i - y_j\|^2)^{-1},$$

which can be interpreted as a *physical system of springs*: attractive forces between points that are close in high dimensions (p_{ij} large), repulsive forces between all other points (q_{ij} large where p_{ij} small).

Decomposition into attractive and repulsive forces. A clean way to read the gradient: split $p_{ij} - q_{ij} = p_{ij}(1) - q_{ij}(1)$ and combine with the kernel to get

$$\frac{\partial C}{\partial y_i} = \underbrace{4 \sum_{j \neq i} p_{ij} q_{ij} Z(y_j - y_i)}_{\text{attractive}} - \underbrace{4 \sum_{j \neq i} q_{ij}^2 Z(y_j - y_i)}_{\text{repulsive}},$$

where $Z = \sum_{k \neq \ell} (1 + \|y_k - y_\ell\|^2)^{-1}$ is the partition function. The attractive term sums over high-dim neighbours of i — typically $O(\text{perplexity})$ contributors — and is cheap. The repulsive term involves all other pairs and is the expensive part.

Remark 7.4: $\text{KL}(P \| Q)$ is asymmetric

The KL divergence $\text{KL}(P \| Q)$ pays a heavy penalty when p_{ij} is large but q_{ij} is small (i.e. a high-dim neighbour appears as a low-dim non-neighbour) and a small penalty for the reverse mismatch. Consequence: *t*-SNE **preserves local structure** (neighbours stay together) at the expense of *global structure* (distant clusters can be placed essentially anywhere).

Optimisation tricks: early exaggeration. A standard implementation runs gradient descent in two phases:

1. **Early exaggeration** (first ~ 250 iterations): multiply p_{ij} by a factor $\alpha = 12$. This makes attractive forces dominate and points coalesce into tight clusters even from a random initialisation, because the repulsive term cannot keep up.
2. **Refinement** ($\alpha = 1$ thereafter): the clusters spread out and individual points find their local positions.

Without early exaggeration, *t*-SNE often gets stuck in poor local minima with overlapping clusters.

Initialisation matters. The original 2008 paper used random initialisation. Kobak and Linderman (2021) showed that initialising the 2-D coordinates from the first two PCA components of the data significantly improves global structure preservation. This is now the default in modern *t*-SNE implementations (`openTSNE`, `scanpy`).

Computational tricks. The naïve gradient is $O(n^2)$ per step — prohibitive beyond $n \sim 10^4$. Two algorithmic advances reduce this:

- **Barnes–Hut t -SNE** (van der Maaten, 2014). The repulsive term is approximated using a quadtree (2-D) or octree (3-D) over the current embedding. Distant cells are treated as “effective points” at their centre of mass — the same trick used in N -body gravitational simulations. Cost per iteration: $O(n \log n)$.
- **FIt-SNE / Fast Interpolation t -SNE** (Linderman, Rachh, Hoskins, Steinerberger, Kluger, 2019). Discretises the embedding onto a grid, computes the repulsive kernel by FFT in $O(n)$ per iteration, and interpolates back. Brings t -SNE to 10^7+ points.

7.2 The mass-of-clusters illusion

To make the asymmetric-KL warning concrete, consider an instructive synthetic example. Take three Gaussian clusters in $\mathbb{R}^{10}$:

- Cluster A : 1000 points around the origin with covariance I_{10} .
- Cluster B : 1000 points around $(20, 0, \dots, 0)$ with covariance $0.1 I_{10}$ (much tighter).
- Cluster C : 1000 points around $(40, 0, \dots, 0)$ with covariance I_{10} .

True geometry: A and C are spread, B is dense; A – B and B – C separations are equal; cluster B is dramatically smaller in volume than A and C .

What t -SNE shows. Three blobs of *equal size*, roughly equidistant — the visual size and inter-cluster distances do *not* match the truth. The repulsive forces equalise inter-point distances regardless of the true scale, and the heavy-tailed kernel inflates the tight cluster B to fill comparable visual space.

Lesson. The relative sizes, separations, and densities of clusters in a t -SNE plot do not reliably reflect the data. Only the *neighbour relationships* (which points cluster with which) are trustworthy.

7.3 Pitfalls and goodness-of-trust

The mathematical structure of t -SNE explains its well-documented *misreading hazards*:

- **Cluster sizes are meaningless.** Two clusters with very different intrinsic sizes will appear with comparable visual diameter in the embedding because the repulsive forces equalise inter-point distances.
- **Inter-cluster distances are meaningless.** The KL objective is dominated by local terms; large empty regions between clusters carry no metric information.
- **Random hyperparameter sensitivity.** Different runs (different initial conditions, perplexities, learning rates) can produce qualitatively different layouts on the same data — sometimes spuriously suggesting cluster structure that is not real.
- **Topology can be broken.** A connected 1D manifold can be split into disjoint clusters if perplexity is too low.
- **Outliers cluster spuriously.** Sparse outliers tend to gather together near the centre of the plot, creating “outlier clusters” that do not represent any real grouping in the data.

Quantitative quality measures. Two metrics formalise how much of the high-dimensional neighbour structure the embedding preserves:

- **Trustworthiness** $T(k)$: penalises low-dim neighbours of y_i that were not high-dim neighbours of x_i . Detects “false positives” (clusters not in the data).
- **Continuity** $C(k)$: penalises high-dim neighbours of x_i that were lost in the embedding. Detects “false negatives” (clusters split apart).

Both metrics range from 0 to 1. Computing trustworthiness and continuity at multiple values of k and inspecting the resulting curves is a more rigorous quality check than visual inspection alone.

Remark 7.5: Practical guidance

Use t -SNE for *exploratory* visualisation only. Recommended workflow (Kobak & Berens, 2019):

1. Pre-reduce to ~ 50 PCs with PCA.
2. Initialise from the first two PCs.
3. Run t -SNE with perplexity adapted to dataset size, e.g. perplexity $\approx n/100$ for very large n .
4. Try multiple perplexities (e.g. 5, 30, 100) and seeds before drawing conclusions.

Distill.pub’s interactive article “*How to Use t-SNE Effectively*” (Wattenberg, Viégas, Johnson, 2016) is required reading.

Remark 7.6: Recent theory: when do t -SNE clusters reflect reality?

Arora, Hu, Kothari (2018) and Linderman & Steinerberger (2019) proved partial recovery guarantees: if data come from k well-separated clusters (with controlled in-cluster and between-cluster distances), then t -SNE provably produces k visible clusters with correct cluster memberships in the limit of large n . The guarantees rely on early exaggeration and on cluster separations larger than a model-dependent threshold. These results validate t -SNE as a clustering visualisation tool — but say nothing about the cluster positions or sizes in the embedding.

7.4 UMAP: a Riemannian-geometry-flavoured alternative

UMAP (McInnes, Healy and Melville, 2018) — *Uniform Manifold Approximation and Projection* — is grounded in two assumptions borrowed from differential geometry and topology:

1. The data is uniformly distributed on a Riemannian manifold.
2. This manifold can be approximated by a *fuzzy simplicial complex* built from local metric neighbourhoods.

The full theoretical justification involves category theory and algebraic topology (the simplicial set adjunction), but the implementation reduces to a concrete graph-based algorithm with three stages: build a weighted graph, define a target similarity in embedding space, optimise.

Why “uniform on a manifold”? The uniform-sampling assumption is critical. Real data is rarely uniform, but UMAP enforces it locally by *re-scaling distances at each point*: each point’s σ_i is chosen so that, locally, the data *appears* uniform. This is the mechanism by which UMAP absorbs density variation into the metric and treats the underlying manifold as if it were uniformly sampled.

The fuzzy graph. For each point x_i , the algorithm finds the k nearest neighbours (default $k = 15$) and assigns each edge a membership weight in $[0, 1]$:

$$w_{ij}^{(i)} = \exp(-\max(0, d(x_i, x_j) - \rho_i)/\sigma_i),$$

where ρ_i is the distance from x_i to its *nearest* neighbour (ensuring the closest neighbour gets weight 1) and σ_i is chosen so that

$$\sum_j w_{ij}^{(i)} = \log_2 k.$$

This is the UMAP analogue of t -SNE’s perplexity equation, again solved by binary search over σ_i .

Remark 7.7: The ρ_i shift and connected components

Subtracting ρ_i before exponentiating guarantees that every point has at least one neighbour with weight 1. This is what prevents UMAP from disconnecting the graph even at very small k : every point remains tied to at least its nearest neighbour with full weight. This is a notable difference from t -SNE, which has no analogous mechanism and is more prone to fragmenting connected manifolds.

The two directional weights $w_{ij}^{(i)}$ (from i ’s point of view) and $w_{ij}^{(j)}$ (from j ’s point of view) need to be combined into a single symmetric weight. UMAP uses a fuzzy set union (the probabilistic “or”):

$$w_{ij} = w_{ij}^{(i)} + w_{ij}^{(j)} - w_{ij}^{(i)}w_{ij}^{(j)}.$$

The interpretation: w_{ij} is the probability that x_i and x_j are connected from *at least one* direction. This is precisely the fuzzy-set generalisation of the indicator function “connected in the k -NN graph”.

Fuzzy simplicial complex (topological viewpoint). The weighted graph above is, in the language of algebraic topology, a *1-skeleton* of a fuzzy simplicial complex. Higher simplices (2-cells, 3-cells, etc.) could in principle be added — they would encode the higher-order topology of the data — but in practice UMAP truncates at 1-simplices because higher orders are computationally expensive and the 1-skeleton already captures most of the manifold structure. The 0- and 1-simplex story is the one that survives in the implementation.

The low-dimensional model. Place points $y_i \in \mathbb{R}^d$ and define a low-dimensional membership using a smooth family

$$\Phi(y_i, y_j) = (1 + a \|y_i - y_j\|^{2b})^{-1}.$$

This generalises the Cauchy kernel of t -SNE: for $a = b = 1$ it reduces exactly to $1/(1 + \|y_i - y_j\|^2)$.

Fitting a, b from `min_dist`. The parameters a and b are not user-set directly; they are fitted from a target `min_dist` that the user does choose. The fit imposes:

$$\Phi(y, y') = \begin{cases} 1 & \text{if } \|y - y'\| \leq \text{min_dist}, \\ \exp(-(\|y - y'\| - \text{min_dist})) & \text{otherwise.} \end{cases}$$

A least-squares regression of Φ on $1/(1 + a\|\cdot\|^{2b})$ over a grid of distances yields specific (a, b) values. For the default `min_dist` = 0.1, the fit gives $a \approx 1.93$, $b \approx 0.79$.

The objective. Optimise the embedding by minimising the binary cross-entropy between the fuzzy graph membership weights w_{ij} and the embedded weights $\Phi(y_i, y_j)$:

$$\mathcal{L}_{\text{UMAP}} = \sum_{(i,j)} w_{ij} \log \frac{w_{ij}}{\Phi(y_i, y_j)} + (1 - w_{ij}) \log \frac{1 - w_{ij}}{1 - \Phi(y_i, y_j)}.$$

The first term is attractive (drawing connected points together) and operates on the edges of the fuzzy graph; the second term is repulsive (pushing all unconnected pairs apart) and is the expensive term.

Negative sampling and stochastic gradient descent. The repulsive term is the bottleneck: a naïve computation is $O(n^2)$ per epoch. UMAP applies *negative sampling* — for each attractive update on edge (i, j) , it draws a small number (~ 5 , controlled by the parameter `negative_sample_rate`) of random non-neighbours k and applies repulsive updates only on those pairs. The expectation of the stochastic gradient is proportional to the true gradient, and the per-epoch cost drops to $O(n \cdot k_{\text{NN}})$, linear in n .

This is essentially the same trick as in word2vec (negative sampling for softmax) and modern contrastive self-supervised learning — and is what makes UMAP scale to 10^6+ points where Barnes–Hut t -SNE struggles.

Optimisation details. The full optimiser uses:

- *Initialisation from the Laplacian eigenmap* of the fuzzy graph (a spectral pre-embedding), which gives a far better starting point than random initialisation.
- Stochastic gradient descent with a linearly decreasing learning rate.
- Default 200 epochs for $n < 10^4$, fewer for larger n .
- Approximate k -NN computed via *NN-Descent* (Dong, Charikar, Li, 2011) — an approximate nearest-neighbour algorithm based on the heuristic that “neighbours of neighbours are likely to be neighbours.”

Key hyperparameters.

Parameter	Default	Effect
<code>n_neighbors</code>	15	Larger $\rightarrow$ more global structure preserved
<code>min_dist</code>	0.1	Larger $\rightarrow$ points spread out more in the embedding
<code>metric</code>	Euclidean	Underlying distance (e.g. cosine for word vectors)
<code>n_components</code>	2	Embedding dimension (UMAP works in any d)
<code>spread</code>	1.0	Scale of the embedding (combined with <code>min_dist</code>)
<code>negative_sample_rate</code>	5	Negatives per positive update
<code>n_epochs</code>	auto	Optimisation epochs

Remark 7.8: Reading the hyperparameters geometrically

`n_neighbors` sets the locality scale: small values ($k = 5$) emphasise very local structure and break manifolds into many pieces; large values ($k = 200$) act like a coarser Laplacian-eigenmap and preserve global shape. `min_dist` sets how tightly points are packed in

clusters: small values give visually dense clusters; large values produce a more uniform spread.

7.5 Variants and extensions of UMAP

UMAP is more of a framework than a single algorithm. Several extensions have become standard tools:

- **Supervised UMAP.** If labels are available, the fuzzy graph can be augmented with a label-similarity weight: same-label pairs get higher w_{ij} . The embedding then becomes a supervised dimensionality reduction tool akin to Linear Discriminant Analysis but non-linear. Useful for feature engineering when training a downstream classifier.
- **Aligned UMAP.** For comparing multiple datasets (e.g. scRNA-seq from different patients), aligned UMAP produces consistent embeddings by sharing coordinates across datasets where points are anchored by external matches.
- **Parametric UMAP** (Sainburg, McInnes, Gentner, 2021). Replaces the per-point coordinates y_i with the output of a neural network $f_\theta(x_i)$. The UMAP loss becomes a training objective for the network’s parameters. Result: a learnable, differentiable embedding that generalises to new points without re-running UMAP — useful in pipelines.
- **DensMAP** (Narayan, Berger, Cho, 2021). Augments the UMAP objective with a term that preserves local density, producing embeddings where dense and sparse regions remain visually distinguishable (counteracting one of t -SNE’s cluster-size pathologies).
- **PaCMAP** (Wang, Huang, Rudin, Shaposhnik, 2021). Uses three types of pairs simultaneously (neighbour pairs, mid-near pairs, further pairs) to preserve both local and global structure better than vanilla UMAP. Empirically often the best off-the-shelf method for global structure.

7.6 Critical perspectives

UMAP’s mathematical pitch — categorical fuzzy simplicial sets, Riemannian manifolds, uniform local sampling — is much more elaborate than what the implementation actually does. Damrich and Hamprecht (2021) and Böhm, Berens and Kobak (2022) analysed the actual loss UMAP minimises and concluded:

- The category-theoretic justification is mostly post-hoc; the algorithm in practice reduces to a particular choice of attract/repel forces in a graph layout problem.
- The negative-sampling implementation does *not* minimise the cross-entropy loss as stated in the paper — it minimises a slightly different objective whose actual attractive/repulsive trade-off is more aggressive.
- Despite the theoretical mismatch, the algorithm works well for the same reason as t -SNE: it is an effective attract/repel graph layout method.

The lesson is not that UMAP is wrong, but that the working explanation of *why* UMAP gives good embeddings is the same as for t -SNE: clever attract/repel dynamics on a k -NN graph, not the topology of fuzzy simplicial sets.

Remark 7.9: *t*-SNE vs. UMAP — empirical differences

Compared to *t*-SNE, UMAP typically:

- **Preserves more global structure.** The cross-entropy objective and the explicit attractive/repulsive split are less aggressive about over-separating clusters.
- **Runs faster** — typically 5–10× — thanks to approximate *k*-NN (via NN-Descent) and negative sampling.
- **Produces tighter, more compact clusters** due to the `min_dist` regulariser.
- **Is more reproducible.** While still stochastic, it is less sensitive to initial conditions than *t*-SNE.
- **Generalises to higher embedding dimensions.** *t*-SNE is essentially restricted to 2D/3D; UMAP can embed into 10D, 50D, or any other target dimension and is sometimes used as a general-purpose feature extractor.

Caveat: UMAP can hallucinate clusters in continuous data (especially with low `n_neighbors`); its global geometry is better than *t*-SNE's but should not be over-interpreted.

Remark 7.10: Choosing between *t*-SNE and UMAP — a decision tree

In practice:

- Want to explore cluster structure quickly on a moderate dataset ($n < 10^5$)? **Either works**; pick what your field uses (genomics → UMAP; older ML papers → *t*-SNE).
- Have $> 10^6$ points? **UMAP** (or PaCMAP).
- Need $d > 3$ embedding dimensions for downstream learning? **UMAP** or PaCMAP — *t*-SNE is impractical.
- Care about local cluster purity above all? ***t*-SNE** with careful tuning.
- Care about global structure preservation? **PaCMAP** $\succ$ UMAP $\succ$ *t*-SNE.
- Need a differentiable embedding for a pipeline? **Parametric UMAP**.
- Need theoretical guarantees? Use a spectral method (Diffusion Maps, Isomap) and accept the lower visual quality.

7.7 The unifying perspective

Both *t*-SNE and UMAP can be viewed as instances of a single framework (Böhm, Berens, Kobak, 2022): *contrastive neighbour embedding*. Each defines a high-dimensional graph encoding “which points should be neighbours,” and minimises a loss that

1. pulls graph-connected points together (attractive force),
2. pushes most other pairs apart (repulsive force),

where the attractive/repulsive trade-off is the key knob. *t*-SNE applies a strong repulsion (heavy-tailed Student-*t* denominator), producing visually segregated clusters. UMAP balances attraction and repulsion through the `min_dist` parameter and binary cross-entropy. Changing these

knobs continuously interpolates between the two methods — and explains why their layouts often look qualitatively similar.

Remark 7.11: Connection to contrastive self-supervised learning

The attract-positives/repel-negatives structure of these objectives is essentially the same as the InfoNCE loss used in *contrastive self-supervised representation learning* (SimCLR, MoCo, CLIP). *t*-SNE and UMAP are, in this sense, early instances of contrastive learning — and recent work (*Neighbour Embedding Networks*, 2022) has unified them with modern deep contrastive methods.

Remark 7.12: Spectral vs. optimisation-based methods

Isomap, LLE, Diffusion Maps, and Laplacian Eigenmaps are *spectral*: they reduce to eigendecomposition of a fixed matrix, which guarantees a global optimum but couples computation tightly to matrix size. *t*-SNE and UMAP are *optimisation-based*: they iteratively refine an embedding by gradient descent (with stochasticity in UMAP via negative sampling), which is more scalable and flexible but introduces local minima. Spectral methods give *distances*; *t*-SNE/UMAP give *cluster structure*. No method is universally better; the right choice depends on dataset size, the question being asked (cluster discovery vs. trajectory inference vs. isometric embedding), and how much theoretical guarantee one needs.

8 Summary and Comparison

Method	Core idea	Strength	Weakness	Scale
PCA	Linear variance	Closed form	Linear only	10^7+
Isomap	Geodesic MDS	Globally isometric	Holes, noise	10^4
LLE	Local linear weights	Sparse, holes OK	Flat regions	10^5
Diffusion Maps	Random-walk geometry	Multiscale, robust	σ, t choices	10^4
Laplacian Eig.	Graph Laplacian	Theory, manifold Δ	Sensitive to σ, k	10^5
<i>t</i> -SNE	KL of similarities	Clusters	Global geometry	10^5
UMAP	Fuzzy simplicial sets	Fast, any dim	Stochastic	10^7+
PaCMAP	3-tier neighbours	Global structure	Newer	10^6+
TwoNN ($\hat{d}$)	Pareto of NN ratios	Intrinsic dim only	N/A	10^7+

Reading the table. The first two columns are the mathematical heart of the method; “Scale” is the order-of-magnitude of n where the method is comfortable on a workstation. TwoNN is included as the standard pre-step for any of the others.

Exercises

1. Why does Euclidean distance fail as a measure of similarity on a Swiss-roll dataset? Sketch the Swiss roll and indicate two points that are far apart geodesically but close Euclidean-wise.
2. In Isomap, what can go wrong if the neighbourhood parameter k is chosen too large? Too small? (*Hint: think about short-circuit edges and disconnected graphs.*)
3. In LLE, why is the constraint $\sum_j w_{ij} = 1$ imposed on the reconstruction weights? What geometric invariance does it enforce?

4. A dataset consists of 1000 points on a circle $S^1 \subset \mathbb{R}^2$, sampled with Gaussian noise. Which of the three classical methods (Isomap, LLE, Diffusion Maps) would you expect to recover the circular structure most robustly? Justify your choice.
5. (*Intrinsic dimension*) Sample $n = 5000$ points uniformly from the unit sphere $S^2 \subset \mathbb{R}^3$. What do you expect:
 - PCA spectrum,
 - TwoNN $\hat{d}$,
 - Levina–Bickel $\hat{d}$ for $k = 10$,

to report? Justify each prediction and explain any disagreement.

6. (*Scale-dependent dimension*) A dataset consists of Gaussian noise of standard deviation $\sigma = 0.01$ added to points on a unit 2-D sphere. Sketch the dimension trajectory $\hat{d}(r)$ as r ranges from very small to very large. Identify the three regimes (noise / manifold / global) and indicate the typical $\hat{d}$ in each.
7. Explain why distances between clusters in a t -SNE plot should not be interpreted quantitatively. Sketch a situation in which t -SNE and Diffusion Maps would give qualitatively different visualisations of the same data.
8. In t -SNE the perplexity parameter controls a soft target for the effective number of neighbours of each point. Describe qualitatively what happens to a t -SNE plot of MNIST as perplexity is swept from 5 to 200. Why might a very low perplexity create the *illusion* of many small clusters?
9. (*Early exaggeration*) Explain why t -SNE uses early exaggeration in the first phase of optimisation. What would happen if the exaggeration factor were also applied throughout the run? What if it were applied only at the end?
10. Compare t -SNE and UMAP from the perspective of the attract/repel trade-off described in the unifying perspective subsection. If you had to embed a million data points into $\mathbb{R}^{10}$ for a downstream classifier, which method would you choose and why?
11. (*UMAP ρ_i shift*) UMAP subtracts ρ_i from the distance before exponentiating, ensuring every point has at least one full-weight neighbour. What would happen to the algorithm without this shift? Reason both about the structure of the fuzzy graph and about the optimisation dynamics.
12. (*Negative sampling*) UMAP approximates the repulsive term by drawing ~ 5 random non-neighbours per attractive update. Why is this a justifiable approximation? What is the relationship between negative sampling in UMAP and in the word2vec skip-gram model?
13. (*Laplacian properties*) Verify directly from the definition $L = D - W$ that for any vector $f \in \mathbb{R}^n$,

$$f^\top L f = \frac{1}{2} \sum_{i,j} W_{ij} (f_i - f_j)^2.$$

Hence conclude that L is positive semi-definite and that its kernel contains the constant vector.

14. (*Number of zero eigenvalues*) Show that the multiplicity of the eigenvalue 0 of the unnormalised graph Laplacian equals the number of connected components of the underlying graph. (*Hint:* use the Dirichlet-energy formula above.) What does this tell you about how to detect disconnected components in your dataset *before* running Laplacian Eigenmaps?

Notes on Markov Matrices, Laplacian Eigenmaps, and the Student- t Distribution

Contents

1 Markov Matrices

A *Markov matrix* (or *stochastic matrix*) is a square matrix

$$P = (p_{ij})$$

whose entries satisfy

$$p_{ij} \geq 0$$

and

$$\sum_j p_{ij} = 1$$

for every row i .

An example is

$$P = \begin{bmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{bmatrix}.$$

Such matrices describe the evolution of probability distributions in a Markov chain:

$$x_{k+1} = x_k P.$$

1.1 Basic Properties

- All entries are nonnegative.
- Every row sums to one.
- The largest eigenvalue is always 1.
- Long-term behavior is determined by the spectrum of P .

2 Eigenvalue Interpretation

Eigenvectors satisfy

$$Pv = \lambda v.$$

The eigenvalues describe how different modes evolve under repeated multiplication by P .

2.1 The Eigenvalue One

Every stochastic matrix has an eigenvalue

$$\lambda = 1.$$

The stationary distribution π satisfies

$$\pi P = \pi.$$

Hence π is a left eigenvector associated with eigenvalue 1.

2.2 Bound on Eigenvalues

All eigenvalues satisfy

$$|\lambda| \leq 1.$$

Therefore every eigenvalue lies inside the unit disk.

2.3 Mixing Rate

Assume

$$1 = |\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots.$$

Then

$$P^n = \mathbf{1}\pi + O(\lambda_2^n).$$

The quantity

$$|\lambda_2|$$

is called the *second largest eigenvalue modulus* (SLEM).

- Small $|\lambda_2|$ implies rapid convergence.
- Large $|\lambda_2|$ implies slow convergence.

2.4 Example

Consider

$$P = \begin{bmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \end{bmatrix}.$$

The characteristic polynomial is

$$\lambda^2 - 1.5\lambda + 0.5.$$

The eigenvalues are

$$\lambda_1 = 1, \quad \lambda_2 = 0.5.$$

Therefore deviations from equilibrium decay as

$$0.5^n.$$

2.5 Negative and Complex Eigenvalues

Negative eigenvalues produce oscillatory convergence.

For example,

$$\lambda = -0.8$$

contributes terms of the form

$$(-0.8)^n.$$

Complex eigenvalues

$$\lambda = re^{i\theta}$$

produce damped oscillations

$$r^n e^{in\theta}.$$

2.6 Spectral Decomposition

If P is diagonalizable,

$$P = VDV^{-1},$$

then

$$P^n = VD^nV^{-1},$$

with

$$D = \text{diag}(1, \lambda_2, \lambda_3, \dots).$$

As $n \rightarrow \infty$, all terms associated with

$$|\lambda_i| < 1$$

vanish, leaving only the stationary component.

3 Laplacian Eigenmaps

Laplacian Eigenmaps are a nonlinear dimensionality reduction method based on graph spectral theory.

Given data points

$$x_1, \dots, x_n,$$

a weighted graph is constructed.

3.1 Weight Matrix

A common choice is the Gaussian kernel

$$w_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{t}\right).$$

The weight matrix is

$$W = (w_{ij}).$$

3.2 Graph Laplacian

Define the degree matrix

$$D = \text{diag}(d_1, \dots, d_n),$$

where

$$d_i = \sum_j w_{ij}.$$

The graph Laplacian is

$$L = D - W.$$

Normalized variants include

$$L_{\text{sym}} = I - D^{-1/2} W D^{-1/2},$$

and

$$L_{\text{rw}} = I - D^{-1} W.$$

3.3 Optimization Problem

The goal is to find coordinates

$$y_i \in \mathbb{R}^m$$

that minimize

$$\sum_{i,j} w_{ij} \|y_i - y_j\|^2.$$

Using matrix notation,

$$\sum_{i,j} w_{ij} (y_i - y_j)^2 = 2y^T L y.$$

This encourages neighboring vertices to remain close.

3.4 Generalized Eigenvalue Problem

The embedding is obtained from

$$L v = \lambda D v.$$

Properties:

- $\lambda_0 = 0$.
- The constant vector is the eigenvector associated with λ_0 .
- The first nontrivial eigenvectors provide embedding coordinates.

For a two-dimensional embedding,

$$x_i \mapsto (v_1(i), v_2(i)).$$

3.5 Connection with Markov Chains

Define

$$P = D^{-1}W.$$

Then

$$L_{\text{rw}} = I - P.$$

Thus the graph Laplacian and Markov transition matrix contain equivalent spectral information.

Small Laplacian eigenvalues correspond to slowly decaying Markov modes.

3.6 Relation to Spectral Clustering

Both spectral clustering and Laplacian Eigenmaps use low-frequency eigenvectors of the graph Laplacian.

- Spectral clustering applies k -means to the eigenvector coordinates.
- Laplacian Eigenmaps use the coordinates directly as an embedding.

4 Student- t Distribution

The Student- t distribution with ν degrees of freedom has density

$$f(x) = \frac{\Gamma((\nu+1)/2)}{\sqrt{\nu\pi}\Gamma(\nu/2)} \left(1 + \frac{x^2}{\nu}\right)^{-(\nu+1)/2}.$$

The parameter ν controls the tail behavior.

5 Heavy Tails

For large values of $|x|$,

$$f(x) \sim C|x|^{-(\nu+1)}.$$

Hence the tails decay polynomially.

By comparison, a Gaussian density decays as

$$e^{-x^2/2},$$

which is exponentially faster.

Consequently, extreme observations occur much more frequently under a Student- t model.

6 Effect of Degrees of Freedom

ν	Behavior
1	Cauchy distribution
2	Infinite variance
5	Heavy-tailed
10	Close to Gaussian
∞	Gaussian limit

Smaller values of ν correspond to heavier tails.

7 Existence of Moments

A useful criterion is

$$E(|X|^k) < \infty \iff k < \nu.$$

Therefore:

Degrees of freedom	Mean	Variance
$\nu \leq 1$	No	No
$1 < \nu \leq 2$	Yes	No
$\nu > 2$	Yes	Yes

8 Scale-Mixture Representation

A Student- t random variable can be represented as

$$X = \frac{Z}{\sqrt{U/\nu}},$$

where

$$Z \sim N(0, 1), \quad U \sim \chi_\nu^2.$$

This interpretation explains the heavy tails: occasional small values of U generate unusually large observations.

9 Applications

Student- t distributions play an important role in

- Robust statistics,
- Bayesian inference,
- Financial modeling,
- Network traffic analysis,
- Machine learning.

Compared with Gaussian models, they are substantially less sensitive to outliers.

10 Connection with t-SNE

The t-SNE algorithm uses a Student- t distribution in the low-dimensional embedding space, typically with

$$\nu = 1,$$

corresponding to the Cauchy distribution.

The heavy tails alleviate the *crowding problem* by allowing distant points to remain separated in the embedding.

11 Summary

- Markov matrices describe stochastic transitions.
- Eigenvalue 1 determines equilibrium behavior.
- The second eigenvalue controls mixing speed.
- Laplacian Eigenmaps use graph Laplacian eigenvectors to uncover manifold structure.
- The graph Laplacian is closely related to Markov transition matrices.
- Student- t distributions exhibit polynomially decaying heavy tails.
- Heavy-tailed models are fundamental in robust statistics and modern machine learning.

15. (*Laplacian Eigenmaps vs Diffusion Maps*) On the same k -NN graph with heat-kernel weights, compute the bottom 5 non-trivial eigenvectors of $L_{\text{rw}} = I - P$ and the top 5 non-trivial eigenvectors of P . How are the two eigenvector sets related? Why is the order reversed? (*Hint: $L_{\text{rw}} = I - P$ means $\lambda_{\text{Lap}} = 1 - \lambda_{\text{Diff}}$.*)
16. (*Challenge*) The Markov matrix P in Diffusion Maps has eigenvalue 1 with eigenvector the all-ones vector. Why is this eigenvector discarded when constructing the embedding? What does it represent geometrically? (Same argument applies to the trivial eigenvector of L — connect the two.)
17. (*Challenge*) The original UMAP paper justifies the algorithm via fuzzy simplicial sets and category theory, but Damrich and Hamprecht (2021) showed the implementation does not actually minimise the stated loss. Read their critique and write a short summary (1 paragraph) of what UMAP really optimises, and why the algorithm still produces useful embeddings.